BEAST

BEAST - Thai Duong, Blogger.

- Published: 2011-09-25
- Original: <https://vnhacker.blogspot.com/2011/09/beast.html>
- Preserved from: https://vnhacker.blogspot.com/2011/09/beast.html (live) on 2026-08-09
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

BEAST

So we gave a talk and a live demo at ekoparty last week to show how BEAST exploits a weakness in SSL to decrypt secret cookies.

Please note that BEAST does not do any harm to remote servers. In fact, no packet from BEAST has ever been sent to any servers. We chose PayPal because they do everything right when it comes to server-side SSL, and that is good to demonstrate the power of BEAST, which is a client-side SSL attack. We reported the vulnerability to browser, plugin and SSL vendors several months ago (CVE-2011-3389).

Current version of BEAST consists of Javascript/applet agents and a network sniffer. We have some choices for the agent. At the time we reported the bug to vendors, HTML5 WebSockets could be used to build a BEAST agent but, due to unrelated reasons, the WebSockets protocol was already in the process of changing in such a way that stopped it. We can't use the new WebSockets protocol shipped with browsers. We use a Java applet in this video, but please be aware that it may be possible to implement a Javascript agent with XMLHttpRequest as well. Why don't you take a look? ;-)

Note that it is relatively easy to run a script or an applet in your browser without you doing anything (e.g, by intercepting any HTTP requests from your browser.) After all, each agent is just a piece of Javascript or an applet. Once an agent has been loaded, BEAST can patiently wait until you sign in to some valuable websites to steal your accounts.

In order to make the Java applet agent work, we have to bypass the same-origin policy (SOP). Some people have gotten the impression that BEAST required an SOP bypass bug to work and so it's not a threat by itself. That's not true. It is well known that even with a SOP bypass in Java, you can't read existing cookies. You can send requests and may read responses (which may include new cookies), but no, you can't read existing cookies. In the video (and the live demo as well,) we show

clearly that we decrypt *existing* cookies that were already stored in the browser's cookie jar. During our research, we indeed found a Java SOP bypass. We wanted to focus on more important parts of BEAST such as the actual crypto attack and optimizations, so we stopped looking for alternatives, and used the SOP vulnerability to make an agent.

A. Shamir (the S in RSA) once said "Cryptography is typically bypassed, not penetrated," and please keep reading if you are curious what happened during our anecdote of penetrating crypto.

It began with the alleged backdoor in OpenBSD's IPSEC stack. One day in late December 2010 Juliano sent me an email telling me that he got a new idea. He was at some beach in Indonesia reading tls-cbc.txt (I know that beach and TLS and CBC should not appear in the same sentence but, well, maybe he's not very interested in bikinis) and he came up with the chosen-boundary attack. In hindsight, it's obvious that somebody would think of that when they read about Dai's attack. In hindsight, however, everything is obvious. It takes somebody like Juliano to have such a good idea. I am so lucky that he always shares his ideas with me. I wrote some test cases (using the wonderful tlslite library), and after some hours, my conclusion was... it can't work in browsers. I then moved to the States, and was busy with new life, new job and schooling, so I didn't have time to research that idea any further, even after Juliano kept asking me to check it again. Don't listen to me if I tell you your attack can't work :-). Don't listen to anybody telling you that.

Fast forward to early April. I was working on some project at Matasano, and some SSL code that I saw that day made me realize that I wrote wrong tests for Juliano's idea. In fact, it seems that I didn't quite understand it back then. As soon as I got back home, I re-read Juliano's email, my notes and scripts. I decided that I needed to make it work with pen and paper first, so I drew some diagrams. The result looked hopeful. I then modified the test cases, re-ran them, and for the first time, I saw that chosen-boundary attack may work. That moment was so wonderful. It turns out that I had to "reverse" the idea to make it "compatible" with browsers, and after some more hours, not only I saw that the attack is possible, but I also understood which conditions I need to make it really work. The conditions looked easy too. I was very excited. I would have screamed loudly hahaha. I took note of what I had seen so far, and sent it to Juliano.

At first, Juliano didn't understand my note (because it's a reverse of his idea,) but he caught up very quickly, and he agreed that it looks doable. So the main condition is to be able to send two SSL records in the same cookie-bearing request (if you've read the leaked draft, we call this the blockwise privilege.) We were both very excited, because at that moment we know that it is just a matter of browser features for us to create a reliable exploit for something that people have thought un-exploitable in years.

I collected a list of browser features and plugins that allow me to send cookie-bearing requests. I took a look at the Browser Security Handbook by Michal Zalewski, and found some candidates: Javascript XMLHttpRequest, Java URLConnection, Flash URLRequest, and Silverlight WebClient API. We really wanted to make the attack work with native Javascript, so we spent a lot of time on XMLHttpRequest object. At some point, Juliano and I were even reading C++ source code of various browsers (which is even harder to understand than assembly as a friend once said.) Maybe we've missed something really obvious, but we've never made XMLHttpRequest work the way we need. It was both surprising and frustrating that it is so hard to open full-duplex channels do request streaming in modern browsers. People have to invent a lot of ugly bitches that known as Comet techniques. I studied every single one of them, but none of them meets what we need. I

also studied HTML5 WebSockets, then concluded that it's not what I need because it includes a \x00 byte in front of every packet. More about WebSockets in a moment.

So I moved on to Java URLConnection. I'd never written an applet before, but researching is doing exactly things that you haven't done before. So I wrote an applet, and tried to make it perform the blockwise step. The Internet is really helpful. I found a wonderful URLConnection mini-guide in Stack Overflow. I also wrote a small SSL server to test my applet. It worked as expected. I could open a request, append more data to it as long as I want, and each time I "flush" the output stream, Java would send out a record in the same SSL connection. So wonderful, but I want more. I want something that works without any plugins. Once again I started writing tests for XMLHttpRequest, reading C++ code, or studying Comet. Rinse and repeat. Nothing worked.

One day while in the shower, I realized that those things haven't worked simply because they can't work. That's why people have to invent WebSockets. Hmm, why couldn't I use WebSockets? I re-read my note. So they have a \x00 prefix, which prevents me from fully controlling the chosen block. I remembered that Bellare et al. also had the same problem when they tried to attack SSH, so I re-read their paper. I was quite disappointed to know that they didn't describe any practical way to solve that problem. Then I came up with the idea of chaining of predictable IV. I wrote a small WebSockets simulator to test it, and it works. Not very fast though, since WebSockets requires that my chosen block to be a valid UTF-8 string. It's funny that we also had to deal with UTF-8 in the ASP.NET exploit. Anyway, it doesn't need any plugins. It was late April.

We split the work. I wanted to work on the paper, and Juliano wanted to work on the exploit. I started writing right away, but Juliano had to delay the exploit several months later. He got a name for it anyway. "This thing is so complicated. I would like to call it B.E.A.S.T so people kind of get stuck calling it 'the BEAST attack'. We can figure out what BEAST means later."

Writing in English has never been easy for both of us. It took us several months with a lot of help from friends to finish the ASP.NET paper, and I couldn't believe that I had to write another one even before releasing that paper. But I did eventually. Along the way, I found Bard's papers, which was extremely helpful for me. I read his paper carefully. So many "We believe". I have to confess that I am a non-believer, so I made a rule for myself: no "We believe" in my paper. Anyway, although Bard's attacks can't work, his papers were written in very nice English. So I stole a lot of phrases, expressions and statements from him. Of course I cited him many times.

So I kept writing, and Juliano helped with editing. By mid May, we finally had something good enough to ask for review from friends. I guess that no "We believe" is a good rule, since everybody said that the paper is easy to understand. We also got an awesome review from Kenny Paterson. If you happen to write a crypto paper, and need some cryptographer to review it, you may want to ask Kenny. He's very friendly, encouraging, and his review always teaches us a lot.

So we got a not-so-bad paper, but still no actual exploit because Juliano was still in some beach somewhere. We agreed that we should contact browser and SSL vendors early so that they can work on the patch, since we planned to (but didn't) release something in July. We sent the paper to Google, Mozilla, Apple, Microsoft, Opera and Oracle. All of them responded very quickly, which is pretty impressing. Mozilla created bug 665814, and it became the discussion board of all people working on the fix. Later I discovered that there were also people from IETF and other vendors that we didn't contact.

It turns out that fixing this is not easy, because every proposed solution is incompatible with some existing SSL applications. OpenSSL has already included a fix several years ago, but it is turned off by default, thanks to Internet Explorer 6's broken SSL implementation. Somebody also pointed out that due to another attack released in March, the WebSockets protocol was already in the process of changing in such a way that stopped our attack. People kept asking for a working PoC, but we could not give them anything. At some point, it seemed that no vendor wanted to patch this. We also started losing interest in convincing them. We got more compliments from cryptographers we contacted.

We didn't release anything in July as planned, since nothing has been fixed. Instead we went to BlackHat, won a pwnie for the ASP.NET bug, delivered a presentation on the attack at Matasano's private dinner. People liked it. We got several follow-up emails and an invitation to present it again at some internal conference of a client. Juliano and some other friends rooted all servers and yet lost their CTF game. Still no actually BEAST. "We can work together on BEAST when I visit you next week", but we ended up drinking all nights. So many hi 5...

Then Juliano submited the talk to ekoparty. Opera patched. We got excited. That was five weeks ago.

Juliano told me that the talk got 10/10 rating from all of the judges, and he felt that it's time to give birth to BEAST. Since WebSockets is not a good option anymore, and there was so little time to research other alternatives, we agreed that we should focus on Java and Silverlight. He worked and worked and worked. It turns out that BEAST is not easy to code. At some point, Juliano had to install Windows and Visual Studio to write a Silverlight applet in, cough cough, VB.NET. Well, he has to do things he's never done before.

BEAST finally decrypted the first byte of some cookies. It is so surreal to witness some idea that exists only in your mind actually evolves into working code. Moments like this are very rewarding, and it makes researching very worth doing. Juliano was so tired, so I asked him to send the code to me. This is why we've enjoyed doing things together. We can make progress as long as there is at least one guy up. He has done the heavy work, now it's my turn to baby-sit BEAST.

I installed Eclipse, learned Java, and started hacking the code. Since Juliano stopped as soon as BEAST decrypts the first byte, I had to fix bugs to make it decrypt more bytes more reliably. The next thing I did was to find a way to bypass the same-origin policy (SOP) with some agent. A friend was so generous that he gave me one of his Java 0-days to bypass SOP. Anyway, that bug has a dependency that we couldn't satisfy, so I had to find another one. I started reviewing JVM's source code. Honestly, I didn't expect to find a SOP bypass, but I did. What interesting is that the bug is very good for BEAST, but not so good for other types of attackers. You need to be able to do MiTM to use it. Well, that makes sense when you know that I found the bug when I was MiTM-ing a Java applet. I could look for other ways to create an agent, but both of us agreed that we should focus on optimizations. I needed to make BEAST fast. The version that Juliano sent me was so slow that all we got were expired cookies. BEAST is just a baby, it deserves fresh cookies. I spent the last week or so working on that. As you see in the demo, BEAST now takes minutes to decrypt very long unexpired cookies.

In summary, we had an idea, and we've done several things we've never done before to make it work. That's our story of penetrating crypto. Thank you for your time.

Thanks to Marsh Ray and Juliano Rizzo for reading and editing drafts of this.